Clock-Based Text Detection in Literature

Alex Taschuk, 80159916

***Abstract*—There are several ways in the English language to depict the current time. For example, a reader can be explicitly told with "*Spencer arrived home at twelve o'clock in the morning,*" or in a less-explicit manner, such as "*Spencer arrived home at the stroke of midnight.*" As such, the task of manually finding clock-related sentences in literature can be quite tedious. Furthermore, using regular expressions (regex) to detect time-based quotes programmatically is not an easy feat due to the inherent difficulty of writing regex and the complex edge cases that would need to be covered. This paper explores the possibility of using deep learning (DL) to detect sentences in literature that explicitly mention the hour and minute of a day that can be found on a clock. Specifically, I compare how roberta2roberta, a fine-tuned RoBERTa model, performs versus a fine-tuned BERT token classifier + GPT-5.4 mini combination. The code is publicly available on a GitHub repository[1].**

I. INTRODUCTION

LAST year for a personal project, I made a clock that tells the time using quotes from books[2]. The clock stores a CSV file containing over three thousand quotes from a large number of books that span a wide range of genres. The clock will parse a row for the next minute, generate an image of the quote, and display it at the top of the minute. For example, a quote from *The Great Gatsby* by F. Scott Fitzgerald[1] that could be displayed at seven o'clock p.m. is "It was seven o'clock when we got into the coupé with [Gatsby] and started for Long Island." Almost every minute of every hour of the day has at least one quote to display[3], but several have multiple. For minutes with multiple quotes, one is chosen at random to be displayed.

Fig 1. A quote the clock could display for 19:00.

Sometimes, it is necessary to include a preceding or succeeding sentence (or both) to provide proper context to the sentence containing the time. For example, consider the following sentence from Stephen King's *Full Dark, No Stars*: "There were only four words: *Tomorrow morning. 2 o'clock.*" The context of this sentence is not clear to the reader by itself. The four words could be words that someone spoke, wrote, or read. However, when we add the preceding sentence, the context becomes significantly more obvious: "Henry held out his hand for the note, which Victoria gave over in exchange for a Sweet Caporal." Now, it is clear to the reader that the four words were written on a note that a character named Victoria traded to someone named Henry for a pack of Sweet Caporals (a brand of cigarettes).

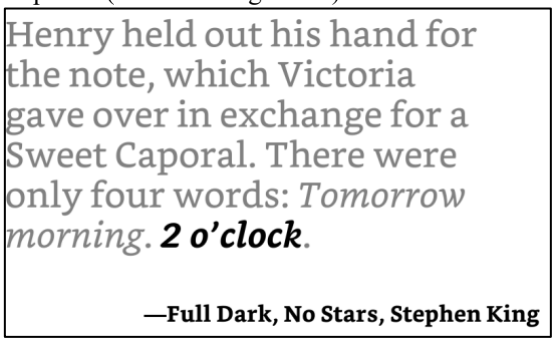

Fig 2. A clock quote whose context is made clear by the preceding sentence.

Adding extra sentences can also enhance the sentence containing the clock quote. Consider this sentence from *The Great Gatsby*: "It was nine o'clock when we finished breakfast and went out on the porch." By itself, this quote is perfectly fine and makes sense. However, we can include the sentence that follows to provide a bit more information to the reader: "The night had made a sharp difference in the weather, and there was an autumn flavor in the air."

In my free time, I enjoy reading and will add quotes to the clock's CSV file as I come across them. Having a large variety of quotes to select from each minute is ideal because it reduces the frequency with which the same quotes are displayed. Manually searching through books to find additional quotes is a very tedious process due to the number of ways that time can be expressed in the English language. Furthermore, the way that time has been expressed in literature has changed throughout history, so using a set list of keywords to search for (e.g., "o'clock", "half-past", "P.M.") would not produce the most ideal or optimal results. I wanted to explore the option of using DL to parse books and identify time-based sentences for adding to the clock. I also wanted the model(s) to be able to determine *when* additional context (i.e., preceding and/or succeeding sentences) was needed or could improve the time-based sentence, and to identify *which* sentence(s) should be included if deemed applicable.

[1] https://github.com/alextaschuk/Time-Based-Quote-Detection

[2] https://github.com/alextaschuk/Literary-Quote-Clock

[3] Currently (in 24-hour time), the clock is missing quotes for 00:39, 00:51, 00:57, 00:58, 01:29, 02:02, 02:03, 05:26, 06:02, 07:36, 07:38, 07:40, 08:01, 08:51, 09:10, 09:16, 09:43, 09:44, 10:16, 10:58, and 18:04.

II. RESEARCH METHODOLOGY

For this project, I used two fine-tuned models from a paper published in 2021 by Satya Almasian et al. titled *BERT got a Date: Introducing Transformers to Temporal Tagging*[2]. In their paper, they research how two transformer models, BERT and RoBERTa, can be fine-tuned to improve their temporal tagging abilities in text. In total, Almasian et al. fine-tuned five models: two sequence-to-sequence (seq2seq)—one RoBERTa and one BERT—and three token classifiers, all of which were fine-tuned BERT models. All five models tag temporal text using TimeML's TIMEX3 tag. TimeML is a markup language intended to annotate temporal events in text, and TIMEX3 is a tag used for marking explicit temporal events.

Almasian et al. published their paper on arXiv in September 2021. The paper concludes that the roberta2roberta model, a seq2seq model, performed the best. Approximately three months later, at the end of January 2022, Almasian withdrew the paper due to "unreliable evaluation results for seq2seq models." Of the five models in the paper, I used one of the fine-tuned BERT token classifiers called Vanilla BERT (VB) and roberta2roberta.

I experimented with two approaches for clock-sentence detection. Both approaches used the same three books as input: *The Great Gatsby*, by F. Scott Fitzgerald, *Frankenstein; or, The Modern Prometheus*, by Mary Shelley[3], and *Moby-Dick; or, The Whale*, by Herman Melville[4]. I chose these books because they are all considered classics in English literature, and therefore have less likelihood that external factors, such as a lack of training data, would get in the way (versus, for example, a lesser-known book). The books were downloaded as .txt files from Project Gutenberg. Before I began filtering the .txt files, I cleaned them up to remove certain things from them, such as Project Gutenberg's license, the table of contents, and other irrelevant text. *Moby Dick* has chapters that include temporal words (*The Great Gatsby* and *Frankenstein*'s chapters don't have titles). For example, one chapter is titled "**CHAPTER 40. Midnight, Forecastle**." In addition to the irrelevant text that was removed from all three books, all chapter titles were also removed from *Moby Dick*.

After cleaning the files, I used the NLTK Python library to tokenize the input files into individual sentences and stored them in a list. This method is preferable to using standard regex because the library can identify complex edge cases in a much cleaner fashion. Using regex to determine whether a character, such as a period, marks the end of a sentence or is part of an abbreviation can be complicated. For example, consider the sentence "Mr. and Mrs. Smith arrived in the U.S. at 3:45 P.M." This sentence should be tokenized as a single sentence, but complex regex would be required to distinguish the period at the end of the sentence from the periods in "Mr.", "Mrs.", "U.S.", and the first period in "P.M."

The first approach uses a combination of Vanilla BERT and OpenAI's GPT-5.4 mini. After the book's sentences have been tokenized, they are given to VB as input. The sentences that VB tags are written to an output JSON file. Each tagged sentence is written to the file as a JSON object with the following keys:

- The sentence's index in the tokenized sentence list.
- The target sentence (the sentence that has been tagged).
- The full context (preceding sentence, target sentence, and succeeding sentence).
- The TIMEX3 tags (type, text, and score).

For example, this is a tagged object from *The Great Gatsby*:

```
{
    "sentence_index": 99,
    "target_sentence": "Tomorrow!" Then
she added irrelevantly: "You ought to see
the baby." "I'd like to." "She's
asleep.",
    "full_context": "Let's go back, Tom.
Tomorrow!" Then she added irrelevantly:
"You ought to see the baby." "I'd like
to." "She's asleep. She's three years
old.",
    "timex3_entities": [
        {
            "type": "DATE",
            "text": "tomorrow",
            "score": 0.9982
        }
    ]
},
```

You may have noticed that the target sentence includes four sentences, rather than one:

1. "Tomorrow!"
2. Then she added irrelevantly: "You ought to see the baby."
3. "I'd like to."
4. "She's asleep."

There are four sentences in target_sentence because NTLK tokenized them into a single element. The library does this because the continuous pattern of a punctuation mark followed by a closing quotation, then an opening quotation (e.g., ." "), is too ambiguous as a sentence boundary signal, so NTLK defaults to keeping the sentences together.

Next, GPT-5.4 mini is used to filter out the non-clock-related sentences. The model was prompted to determine whether a specific clock time is present and the minimal excerpt that is needed from `full_context` to make the sentence meaningful, along with the n^{th} JSON object from VB's output file (JSON objects were input one at a time). The filtered results were written to an output JSON file in the same format as VB's output file.

The second approach uses roberta2roberta. Almasian et al. concluded that this model was the strongest and performed the

best in their testing (however, as noted earlier, the paper was withdrawn due to unreliable results for this model), so I wanted to see how it would perform by itself against the Vanilla BERT + GPT-5.4 combination. Even though roberta2roberta was supposed to be a strong model, I suspected that it would perform worse because of the strict filtering that would be required without a large language model (LLM). The tagged sentences were written to a file as a JSON object with the following keys:

- The sentence's index in the tokenized sentence list.
- The target sentence (the sentence that has been tagged).
- An annotated version of the target sentence with the tagged temporal text wrapped in TIMEX3 tags.
- The full context (preceding sentence, target sentence, and succeeding sentence).
- The TIMEX3 tags that relate to time in the target sentence, and which text was tagged.
- All of the TIMEX3 tags in the target sentence, and the text that was tagged.

Here is a valid object from *Moby Dick*:

```
{
    "sentence_index": 5995,
    "target_sentence": "By midnight the
works were in full operation.",
    "annotated_sentence": "By <timex3
type=\"TIME\" value=\"1998-12-08T24:00\">
midnight </timeXX </timesx 3>  the works
were in full operation.  . <TIME\"[ value
\"1998\"> noon </x2> .",
    "full_context": "It smells like the
left wing of the day of judgment; it is
an argument for the pit. By midnight the
works were in full operation. We were
clear from the carcase; sail had been
made; the wind was freshening; the wild
ocean darkness was intense.",
    "clock_entities": [
        {
            "clock_time": "24:00",
            "text": "midnight"
        }
    ],
    "all_timex3_entities": [
        {
            "value": "1998-12-08T24:00",
            "text": "midnight"
        }
    ]
},
```

An interesting note is that it seems roberta2roberta partially hallucinated an additional temporal tag, because it randomly added a tag for "noon", a word that is not in the target sentence: `<TIME\"[ value \"1998\"> noon </x2>` I suspect that this is due to the model's lack of training on prose from the time period that *Moby Dick* was published.

After a sentence was tagged and a JSON object was created for it, a regex is used to check if the target sentence has an opening tag in the form of `(T HH:MM)`. This severely limits the number of tagged sentences that are returned by roberta2roberta, but it is an inherent consequence of the approach not using an LLM like GPT-5.4 mini to filter out results.

III. RESULTS

Both approaches were run locally on my MacBook Pro to collect the results. The laptop has an M4 Pro chip and 24GB of RAM. When comparing the two approaches, I was interested in a few things. I wanted to know which approach was the fastest, most accurate, and tagged the most quotes relative to the total number of correct quotes.

TABLE I
Vanilla BERT Output

Book	Processing Time	Number Quotes Tagged
The Great Gatsby	33.1s	514
Frankenstein	43.6s	634
Moby Dick	1m 47.9s	1,479

TABLE II
GPT-5.4 mini Post-Filtering Output

Book	Proc Time	Number Quotes	Number Correct	Number Incorrect	Number of Dupes
The Great Gatsby	6m 10.4s	64	62	2	19
Frankenstein	7m 50.0s	33	32	1	10
Moby Dick	19m 1.4s	51	47	4	5

The Number of Dupes column in TABLE II counts the number of clock sentences that appear in more than one JSON object in the output file. GPT was sometimes given the same sentence more than once because some sentences tagged by VB had a preceding and/or succeeding sentence that was also tagged. For example, GPT is given a JSON object whose `target_sentence` is a valid clock sentence. The object's `full_context` contains a succeeding sentence that includes a word or set of words flagged by VB. Then, on the next iteration, GPT is given the JSON object whose `target_sentence` is the previous input's succeeding sentence. Although `target_sentence` does not have a valid clock sentence, because GPT looks at both the target sentence and the full context to determine if there is a valid clock sentence, it sees the previous JSON object's `target_sentence` in the current object's `full_context` as a preceding sentence, so it determines that both objects contain valid clock sentences.

TABLE III
roberta2roberta Output

Book	Proc Time	Number Quotes	Number Correct	Number Incorrect
The Great Gatsby	55m 15.4s	22	16	6
Frankenstein	76m 29.0s	9	6	3
Moby Dick	209m 24.0s	34	27	7

The Number Correct and Number Incorrect columns were evaluated differently for Table II and Table III. For Table II, because GPT used both `target_sentence` and `full_context` to determine whether an input was valid, the JSON object would be marked correct if either the `target_sentence` *or* the `full_context` in the output contained a clock-related sentence. For example, this is an object in GPT's output file for *Moby Dick* that was incorrectly tagged:

```
{
    "sentence_index": 2606,
    "target_sentence": "Eight bells
there, forward!",
    "full_context": "While the bold
harpooner is striking the whale! MATE'S
VOICE FROM THE QUARTER-DECK. Eight bells
there, forward!",
    "timex3_entities": [
        {
            "type": "DATE",
            "text": "the quarter",
            "score": 0.8946
        }
    ]
},
```

For Table III, only the `target_sentence` of each output object was evaluated since roberta2roberta does not take surrounding sentences into consideration. So, if an object's `target_sentence` does not contain any clock-related text, it is marked incorrect.

IV. DISCUSSION

It is clear from the tables that the Vanilla BERT + GPT approach performs best. It was faster than roberta2roberta, found more quotes, and was more accurate overall. Additionally, I made an interesting observation during my research. The roberta2roberta model tagged a fair number of sentences that VB overlooked or that GPT filtered out.

TABLE IV
Indices of roberta2roberta Tags Missing from Approach #1

Book	Missed by Vanilla BERT	Filtered Out by GPT
The Great Gatsby	1047, 2131	N/A
Frankenstein	N/A	N/A
Moby Dick	3989, 6261, 7032, 7855	162, 3517, 6056

I found these results quite interesting. Both of the tags from The Great Gatsby that VB missed were for specific "o'clock" times ("eleven o'clock" and "five o'clock"). I looked into this further and realized that VB didn't tag any sentence containing an "o'clock" time. However, those times were still present in GPT's output because other temporal language was often nearby. For example, consider this JSON object from VB's output for *The Great Gatsby*:

```
{
    "sentence_index": 2256,
    "target_sentence": "‟The funeral's
tomorrow," I said.",
    "full_context": "They were hard to
find. ‟The funeral's tomorrow," I said.
‟Three o'clock, here at the house.",
    "timex3_entities": [
        {
            "type": "DATE",
            "text": "tomorrow",
            "score": 0.9964
        }
    ]
},
```

Although VB tagged this sentence because of the word "tomorrow," GPT recognized that "three o'clock" was present in the full context, so it deemed the object valid. The sentences that VB missed and GPT filtered out from *Moby Dick* show no clear explanation for why they were missed or filtered.

TABLE V
Sentences Correctly Tagged by RoBERTa that VB Incorrectly Filtered Out

Index	Description / Context	Tagged Word(s)
3989	Imagery for an engraving of a calm scene depicting an anchored French whaler. The nonce compound is "noon-scene".	"noon"
6261	A ship captain recalls an encounter with Moby Dick.	"midday"
7032	A dead whale was retrieved to the ship before noon.	"ere noon"
7855	The ocean's waves calm as a ship approaches Moby Dick. The nonce compound is "noon-meadow".	"noon"

Three of the four tags VB missed were for "noon," two of which were part of nonce compound words that included "noon." A nonce word is "a word invented for a particular occasion or situation."[5] My guess is that roberta2roberta can split hyphenated words such as "noon-scene," whereas VB cannot, so VB fails to recognize the nonce compounds as temporal language.

Furthermore, I am surprised that VB did not tag "ere noon" because it did tag two sentences where the tagged text was not "noon" by itself (index 7089, "this day noon," and index 8173, "the second the noon"). The last missed tag was for "midday," and I suspect this may be due to a lack of training for the model.

TABLE VI
Sentences Correctly Tagged by RoBERTa that GPT Incorrectly Filtered Out

Index	Description	Tagged Word(s)
162	The narrator describes a painting's subject, a storm at midnight in the Black Sea	"midnight"
3517	A whale's spout is seen from the ship at midnight.	"This midnight"
6056	Finishing the final stages of processing a whale's oil at midnight.	"midnight"

There is no clear reason GPT filtered these out, since it left many sentences whose tagged text was "midnight" in. I believe these three may have been excluded because GPT interpreted "midnight" as a color (e.g., "midnight black") rather than as a time of day.

There is limited research on temporal tagging in literature, and I think there is room for growth and improvement. Moving forward, it may be worth experimenting with different or modified prompts for GPT to filter the data, such as providing the book's title or valid and invalid sentences from the book as examples of what it should and should not look for. There are also stronger LLM models available, and it would be worth testing how they perform. Another optimization that may be worth exploring is using a stronger LLM to also determine the time of day (e.g., 6:00 vs. 18:00). Furthermore, fine-tuning Vanilla BERT on fictional literature to identify clock-related text could be worthwhile to explore.

Using a transformer model for temporal tagging and an LLM for filtering may be useful in other cases where temporal tagging matters, such as improving the accuracy and precision of text summarization. Additionally, I am interested in exploring how this project could be applied to other types of clocks, such as one that tells you the time of day (sunrise, dusk, sunset, twilight, etc.)

V. CONCLUSION

In all, this was an interesting project, and I enjoyed exploring a niche use case for transformer models. The Vanilla BERT + GPT approach was the clear winner among the two approaches I used, but I think there is room for improvement in several ways. That said, the project was a success, and I have created a useful resource for finding new quotes to add to my literary quote clock.

VI. REFERENCES

[1] F. S. (Francis S. Fitzgerald, The Great Gatsby. Project Gutenberg, 2021. Available: https://www.gutenberg.org/ebooks/64317
[2] S. Almasian, D. Aumiller, and M. Gertz, "BERT got a Date: Introducing Transformers to Temporal Tagging," arXiv.org, 2021. https://arxiv.org/abs/2109.14927
[3] M. W. Shelley, Frankenstein; Or, The Modern Prometheus. Project Gutenberg, 1993. Available: https://www.gutenberg.org/ebooks/84
[4] H. Melville, Moby Dick; Or, The Whale. 2001. Available: https://www.gutenberg.org/ebooks/2701
[5] Cambridge Dictionary, "nonce word," @CambridgeWords, Mar. 23, 2022. https://dictionary.cambridge.org/dictionary/english/nonce-word